Introduction to Digital Logic Design Lab
EECS 31L

Lab #01

Student Name: STEVEN KUZHIPALA

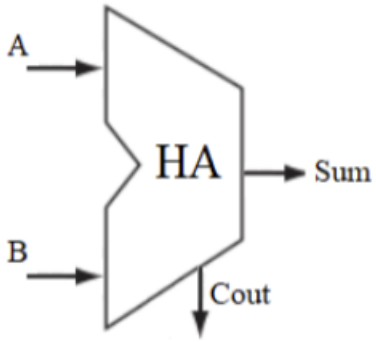
Student ID #: 51378962

Date: 23/1/26

Half Adder

1.Objective:

Create a Half-Adder based of a simple block diagram provided in the lab manual.



The Half-Adder takes in only two inputs, compared to the 3 in the full adder, combines them, and outputs a sum(Sum) and a carryout (Cout).

The logic relation between the inputs A,B and the outputs Cout and Sum is:

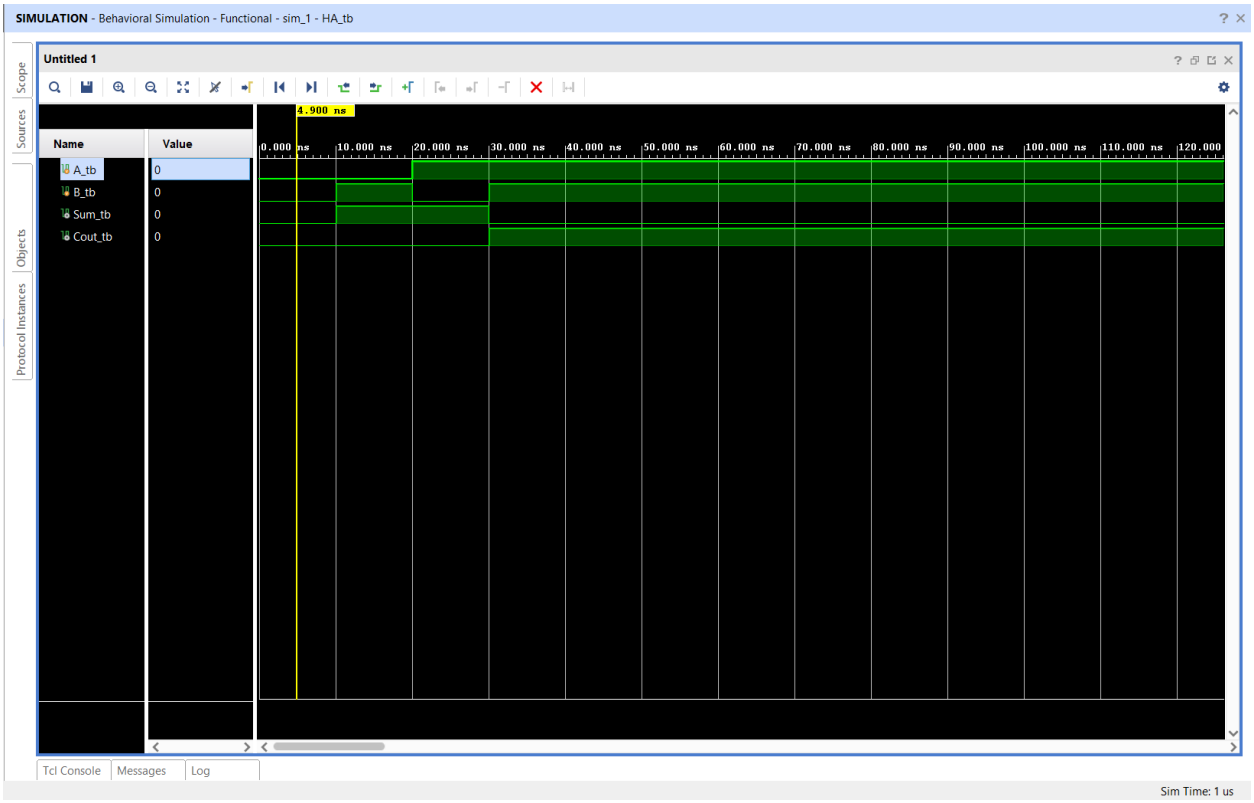
- Sum = A XOR B
- Cout = A AND B

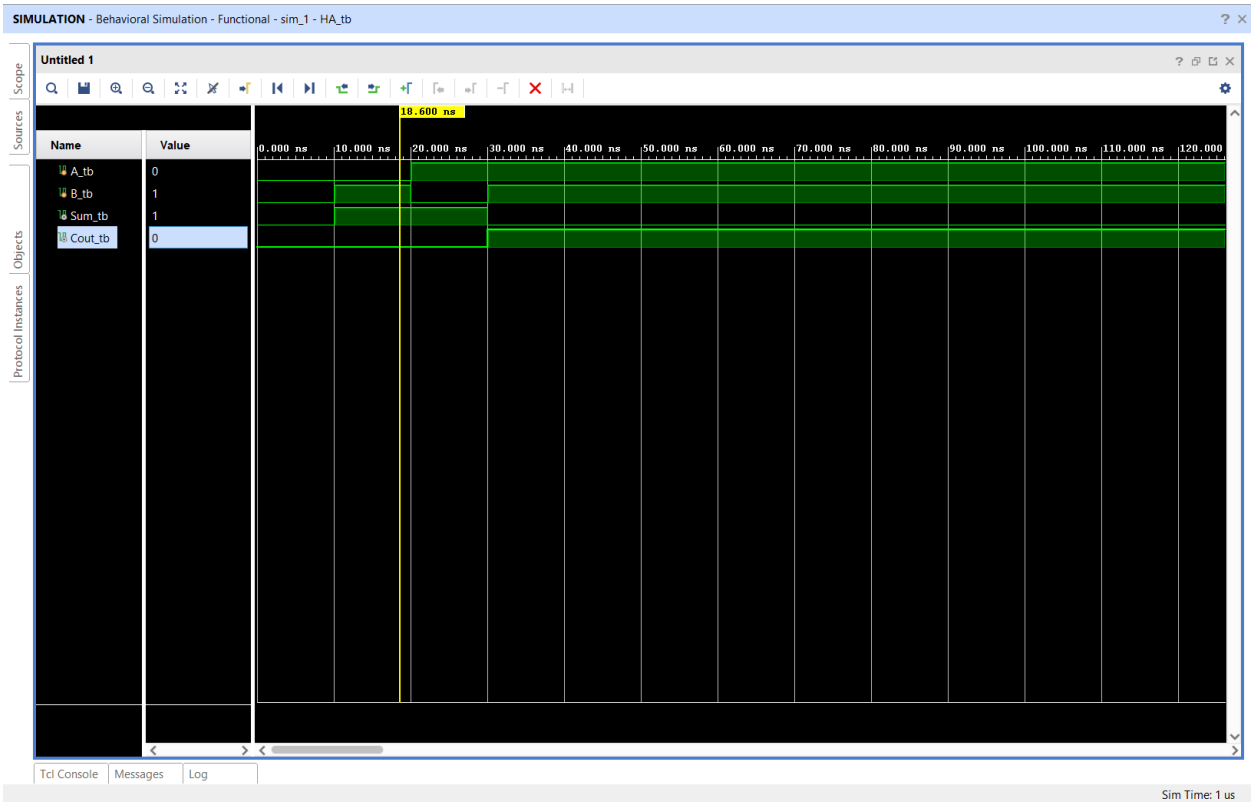
2.Procedure:

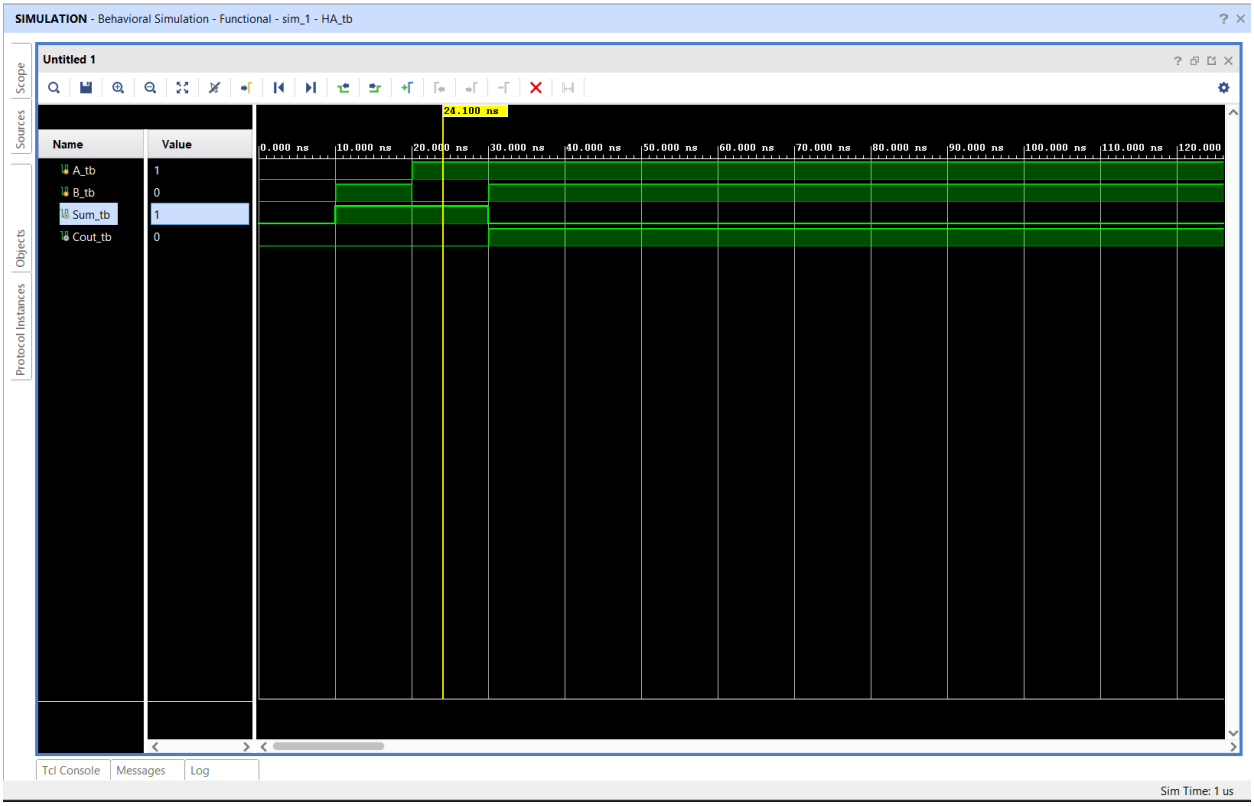
The logic relationship for the Half-Adder could be found from the truth table for the full adder as given below:

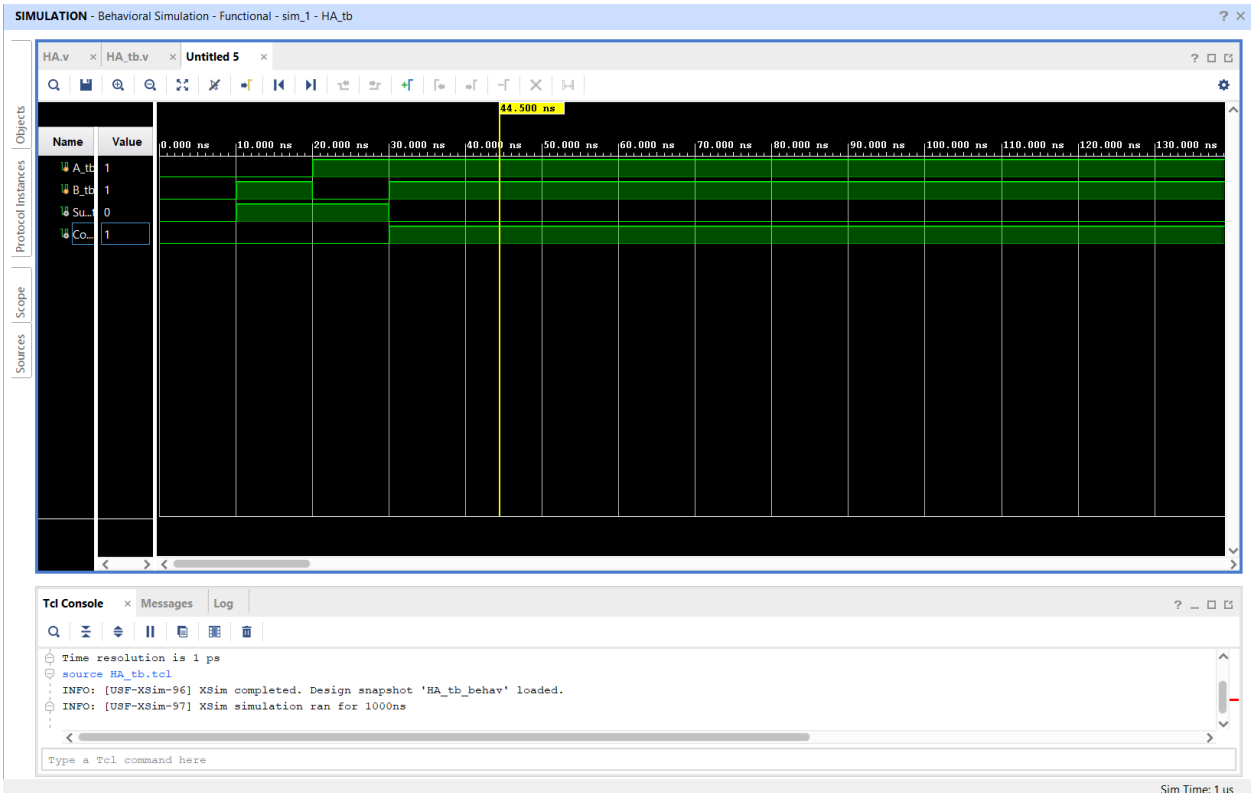
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

3.Simulation Results:




SIMULATION - Behavioral Simulation - Functional - sim_1 - HA_tb
Untitled 1
Scope
Sources
Objects
Protocol Instances
Name
Value
A_tb
B_tb
Sum_tb
Cout_tb
0
1
1
0
18.600 ns
0.000 ns
10.000 ns
20.000 ns
30.000 ns
40.000 ns
50.000 ns
60.000 ns
70.000 ns
80.000 ns
90.000 ns
100.000 ns
110.000 ns
120.000
Tcl Console
Messages
Log
Sim Time: 1 us


SIMULATION - Behavioral Simulation - Functional - sim_1 - HA_tb
Untitled 1
Scope
Sources
Objects
Protocol Instances
Name
Value
A_tb
B_tb
Sum_tb
Cout_tb
1
0
1
0
24.100 ns
0.000 ns
10.000 ns
20.000 ns
30.000 ns
40.000 ns
50.000 ns
60.000 ns
70.000 ns
80.000 ns
90.000 ns
100.000 ns
110.000 ns
120.000
Tcl Console
Messages
Log
Sim Time: 1 us

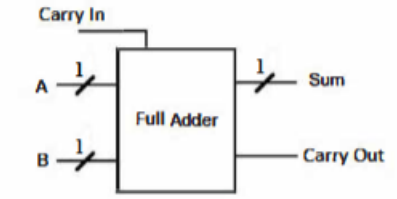


The test cases for the Half-Adder included all the possible inputs for input ports A,B and their expected outputs did match the Truth table discussed earlier.

1-bit Full Adder

1.Objective:

Create a Full-Adder based of a simple block diagram provided in the lab manual.



The 1-Bit Full Adder takes in 3 inputs, the two numbers and the carry-in compared to only 2 from the Half Adder.

The 1-Bit Full Adder we have created here uses the following logical relations to map the inputs to the Sum and Carry out wires:

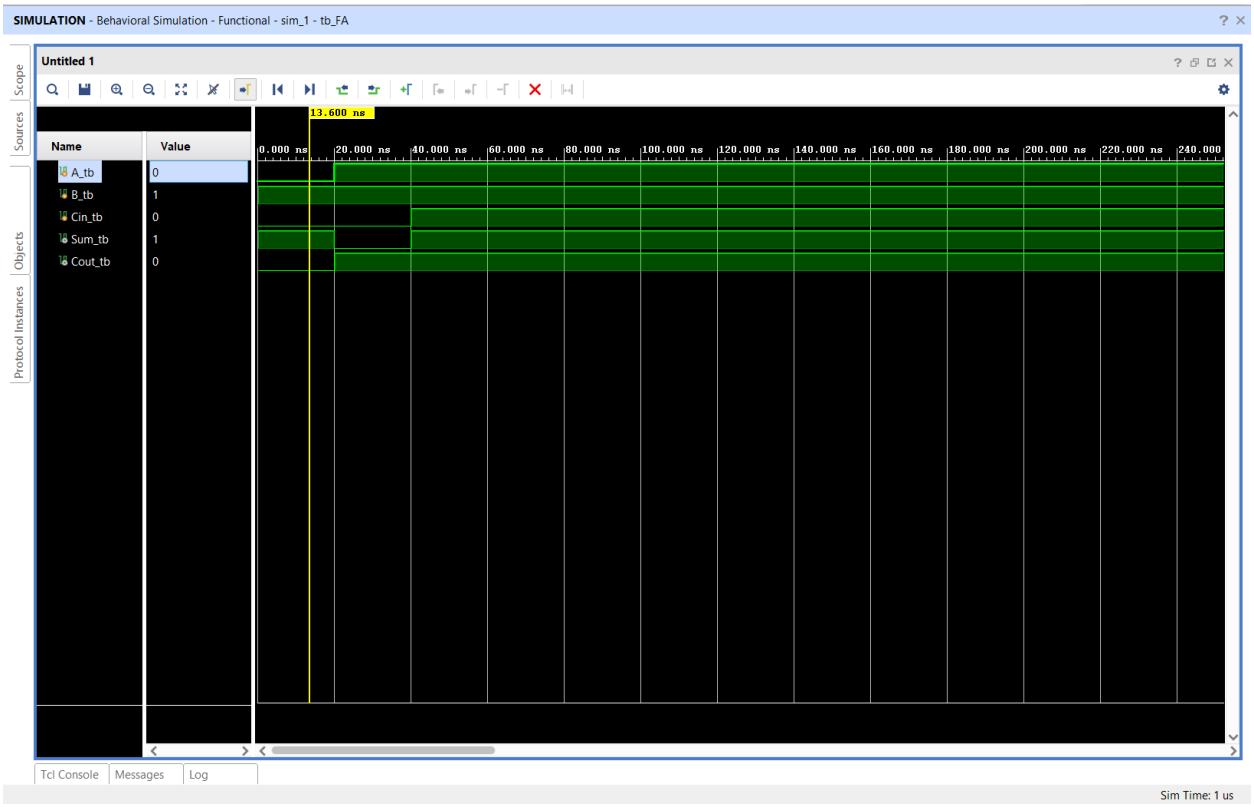
```
Sum = A^B^Cin;
Cout= (A&B) | (A&Cin)| (B&Cin);
```

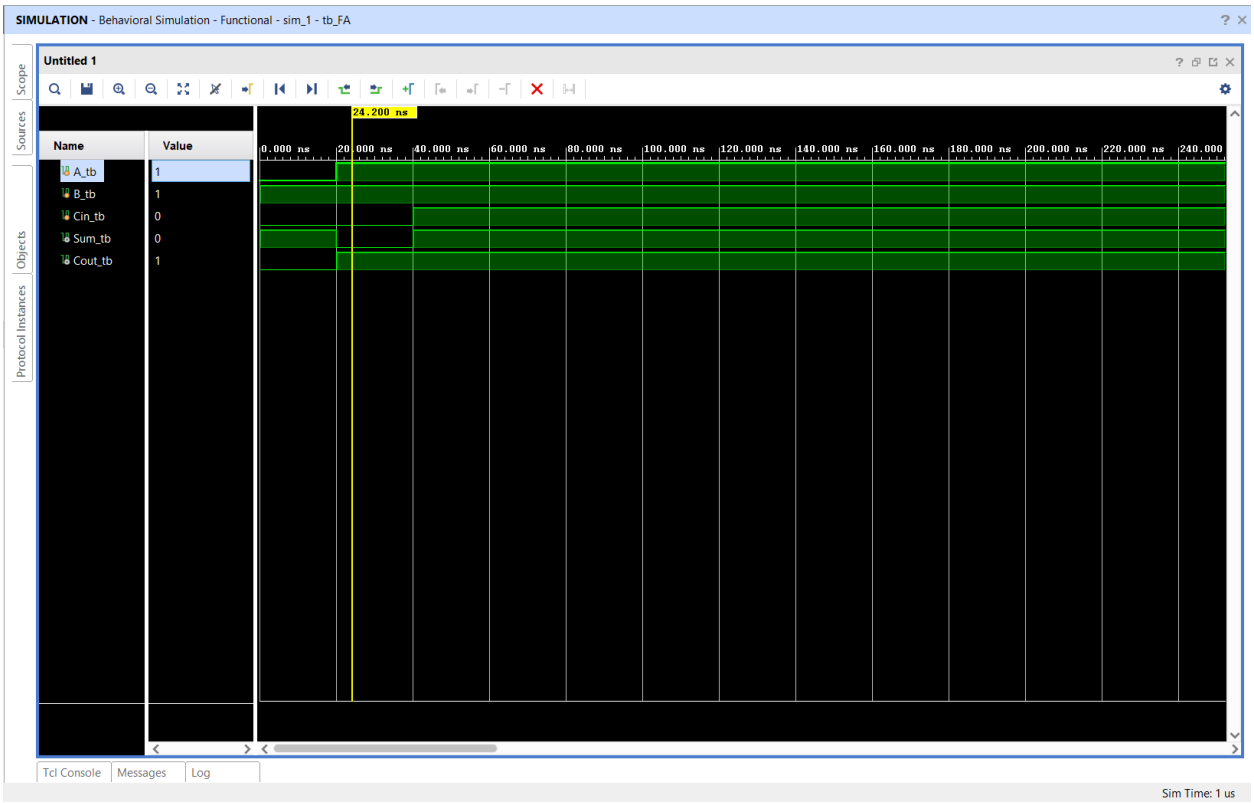
2.Procedure:

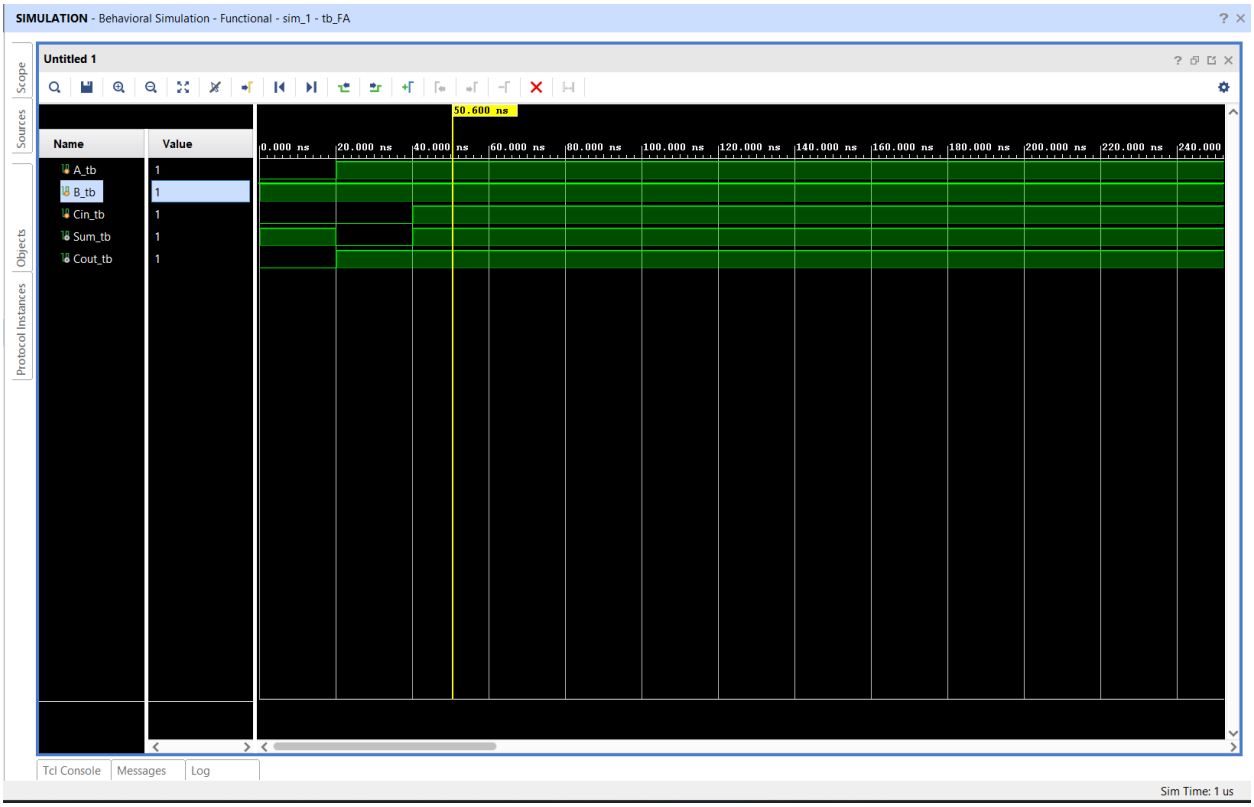
The Truth table for the 1-Bit Full Adder is given below:

Full Adder –Truth Table				
Input			Output	
A	B	Carry in	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3.Simulation Results:





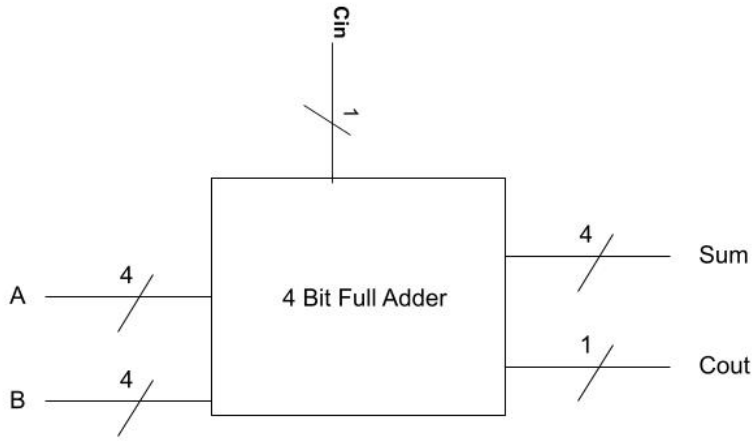


The test cases selected of the simulation included those required by the Lab and included cases that were not applicable for the Half-Adder created previously.

4-bit Full Adder

1.Objective:

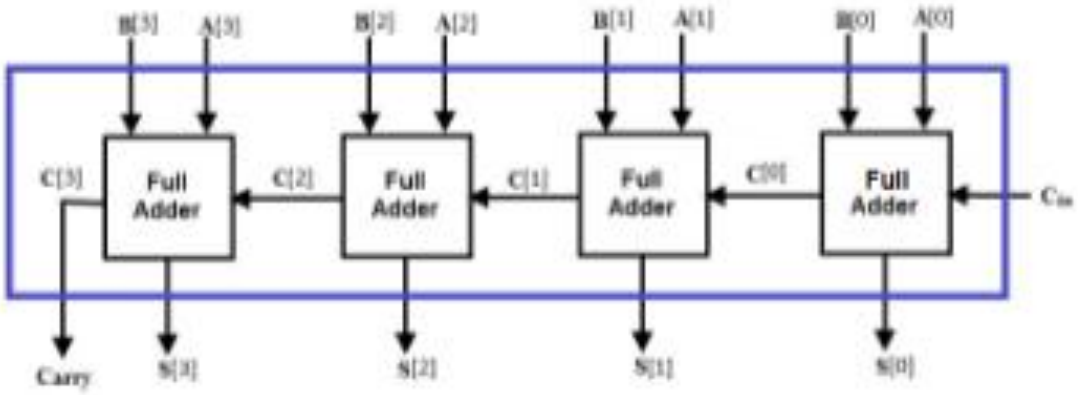
Create a 4 Bit Full-Adder by using multiple instances of the 1-Bit full adder and connecting the Sum wires with each other and producing a 4 bit array as a Sum output.



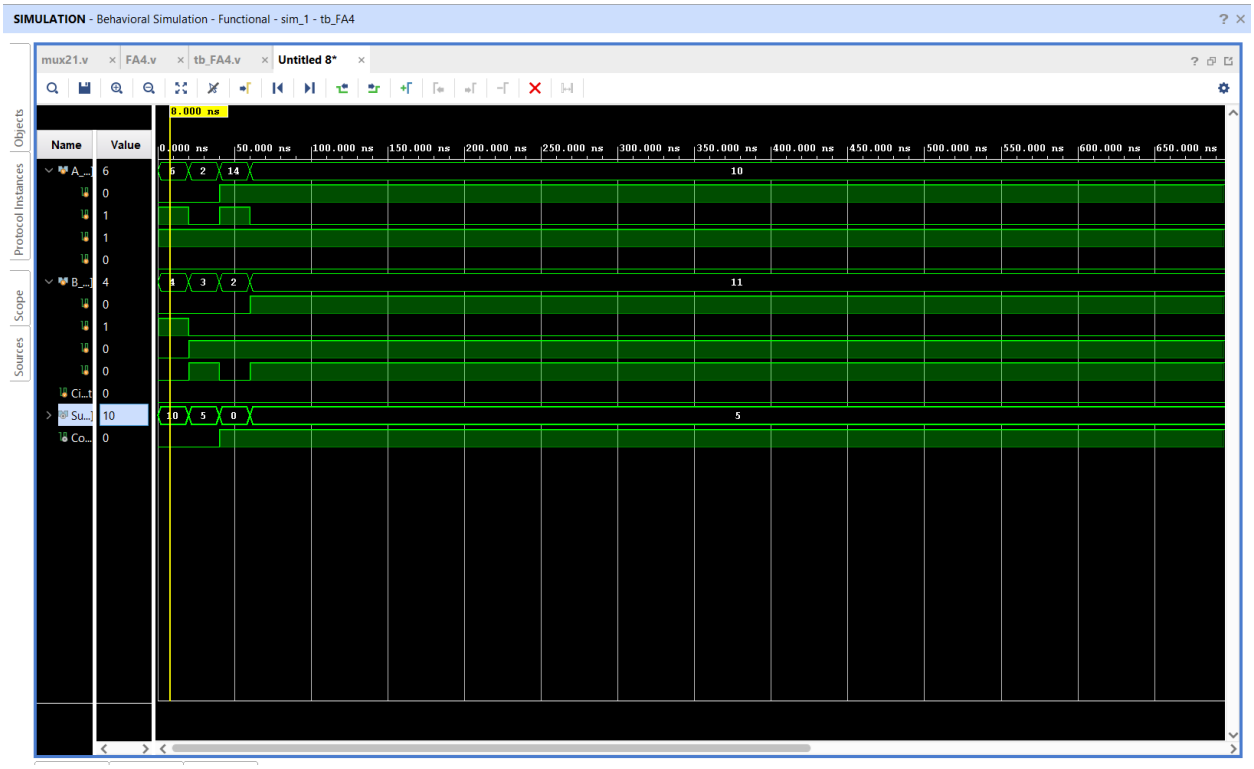
By using 4 instances of the 1-Bit Adder, and connecting each of the individual Full Adders Carry out's as the Carry in's of the next adder (except the last) we are able to expand our 1Bit full Adders to perform addition for 4 bits, or more if needed.

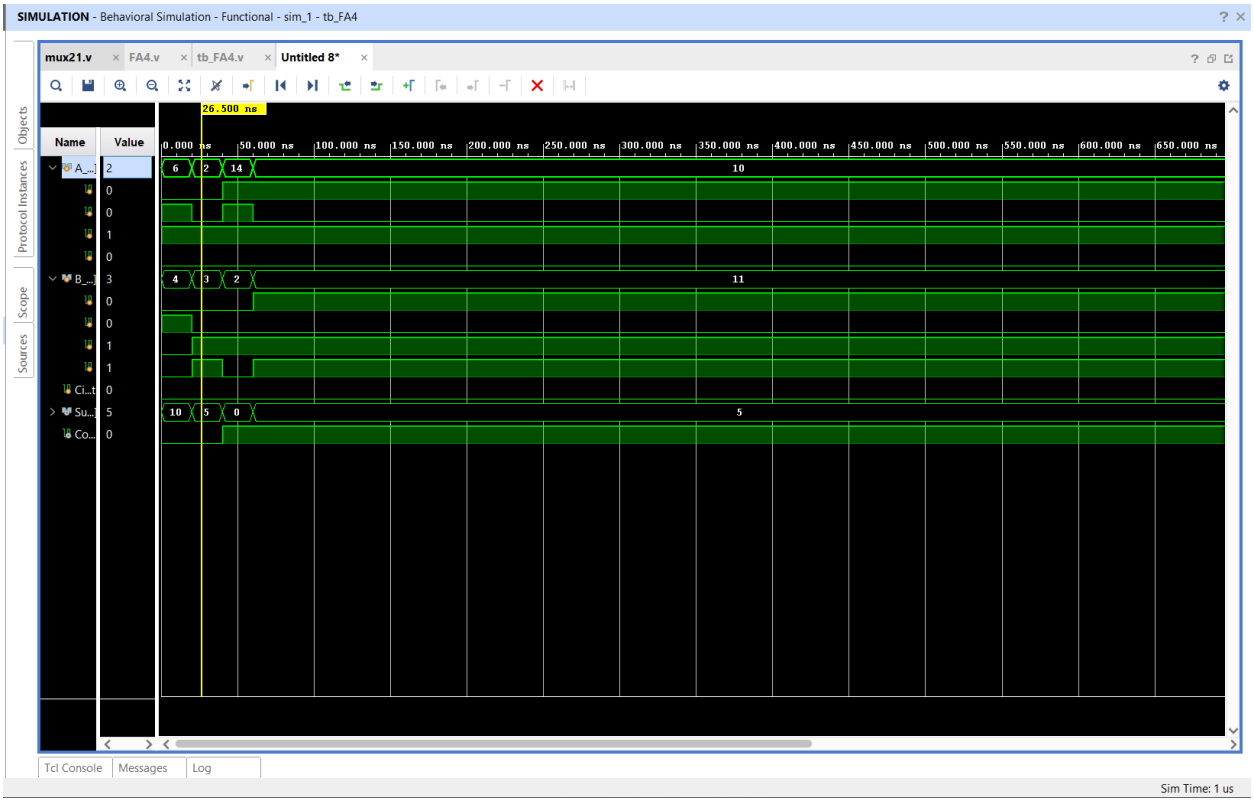
2.Procedure:

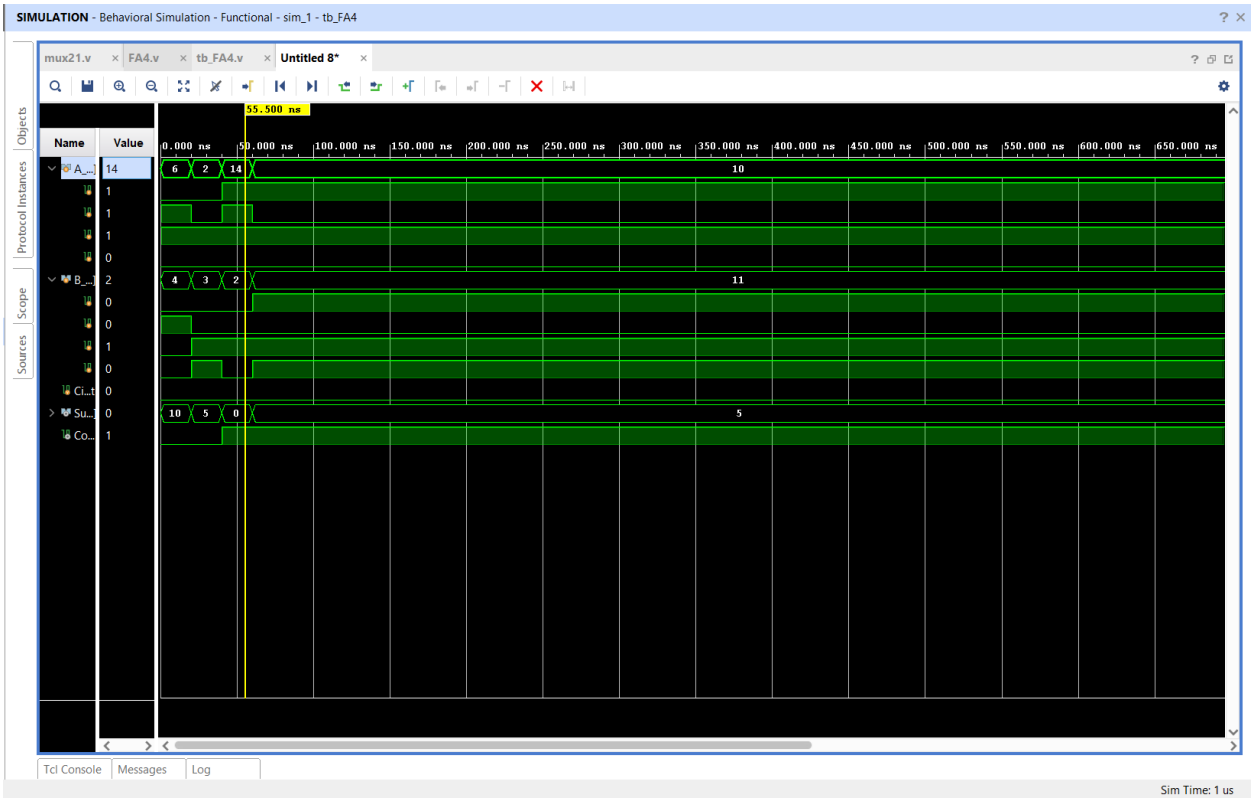
The schematic for the 4 Bit Adder created from the 1 Bit Adder looks as follows, with each of the 1-Bit Full Adders following the same logical relations for their individual SUM and Carryout wires, and the truth table as well.

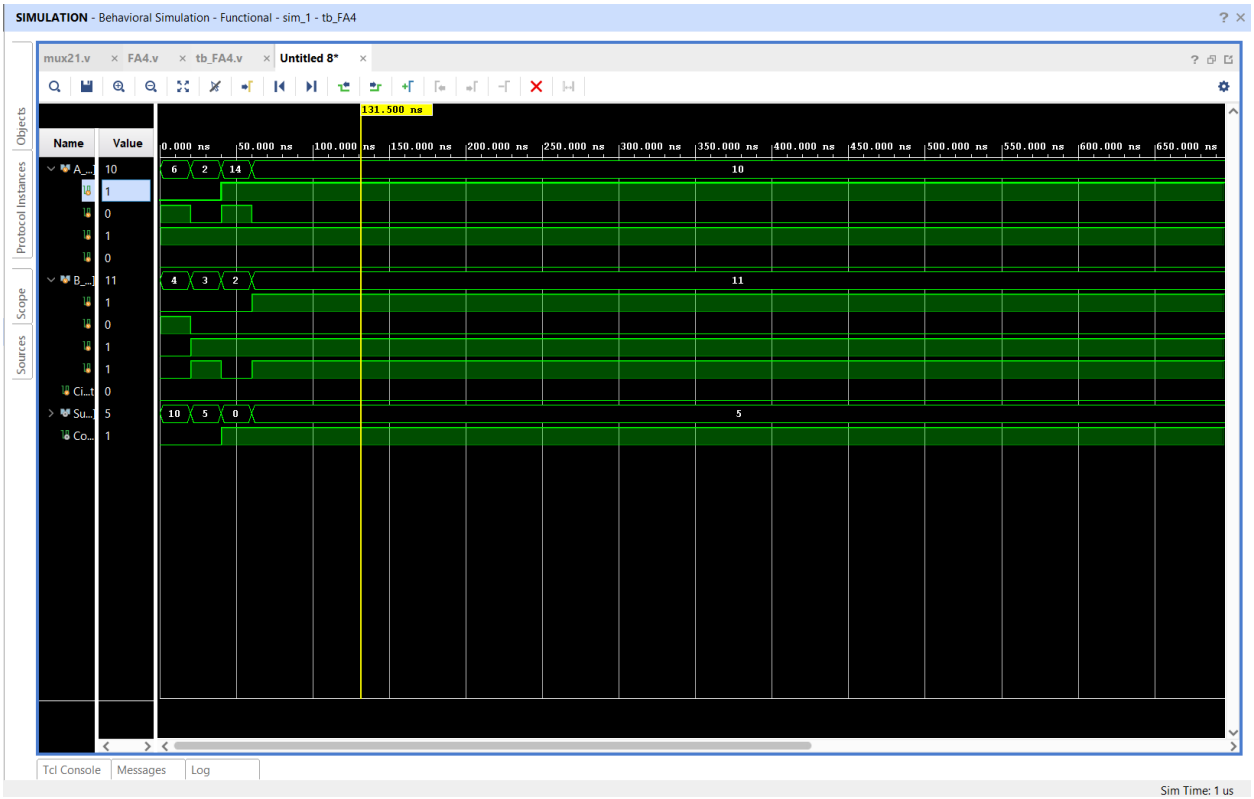


3.Simulation Results:




SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_FA4
mux21.v
FA4.v
tb_FA4.v
Untitled 8*
26.500 ns
Name
Value
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns
450.000 ns
500.000 ns
550.000 ns
600.000 ns
650.000 ns
Tcl Console
Messages
Log
Sim Time: 1 us


SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_FA4
mux21.v
FA4.v
tb_FA4.v
Untitled 8*
55.500 ns
Name
Value
0.000 ns
50.000 ns
100.000 ns
150.000 ns
200.000 ns
250.000 ns
300.000 ns
350.000 ns
400.000 ns
450.000 ns
500.000 ns
550.000 ns
600.000 ns
650.000 ns
Tcl Console
Messages
Log
Sim Time: 1 us



The testbench for the 4 Bit Full Adder included all the tested required by the Lab Manual, and with the successful mapping of the ports to the right array with suitable internal wires to connect a full adders carry out to the next one's carry in (except for the last) as shown below, the outputs were correct and as expected:

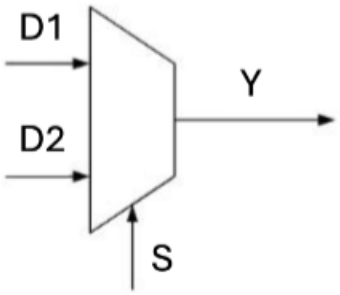
```
//defining all internal carryIn wires, along with final Cout wire
wire Carry_0;
wire Carry_1;
wire Carry_2;

//Port_Mapping
FA instant_0
(
    .A(A[0]),
    .B(B[0]),
    .Cin(Cin),
    .Sum(Sum[0]),
    .Cout(Carry_0)    //Cout signal being mapped to Carry_0
 );
```

Multiplexer 2:1

1.Objective:

Create a 1 Bit , 2: 1 Multiplexer, that can act as a switch between two possible 1 Bit inputs, following the Block diagram as shown below:

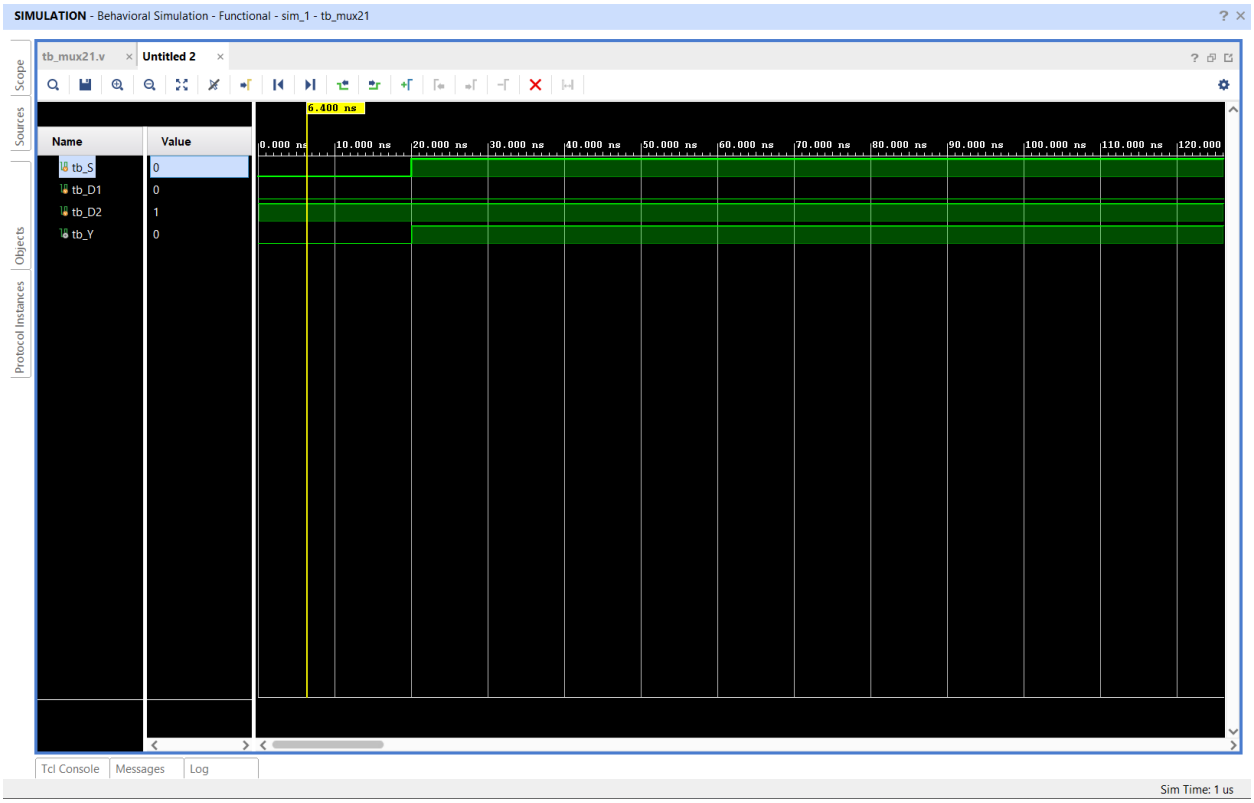


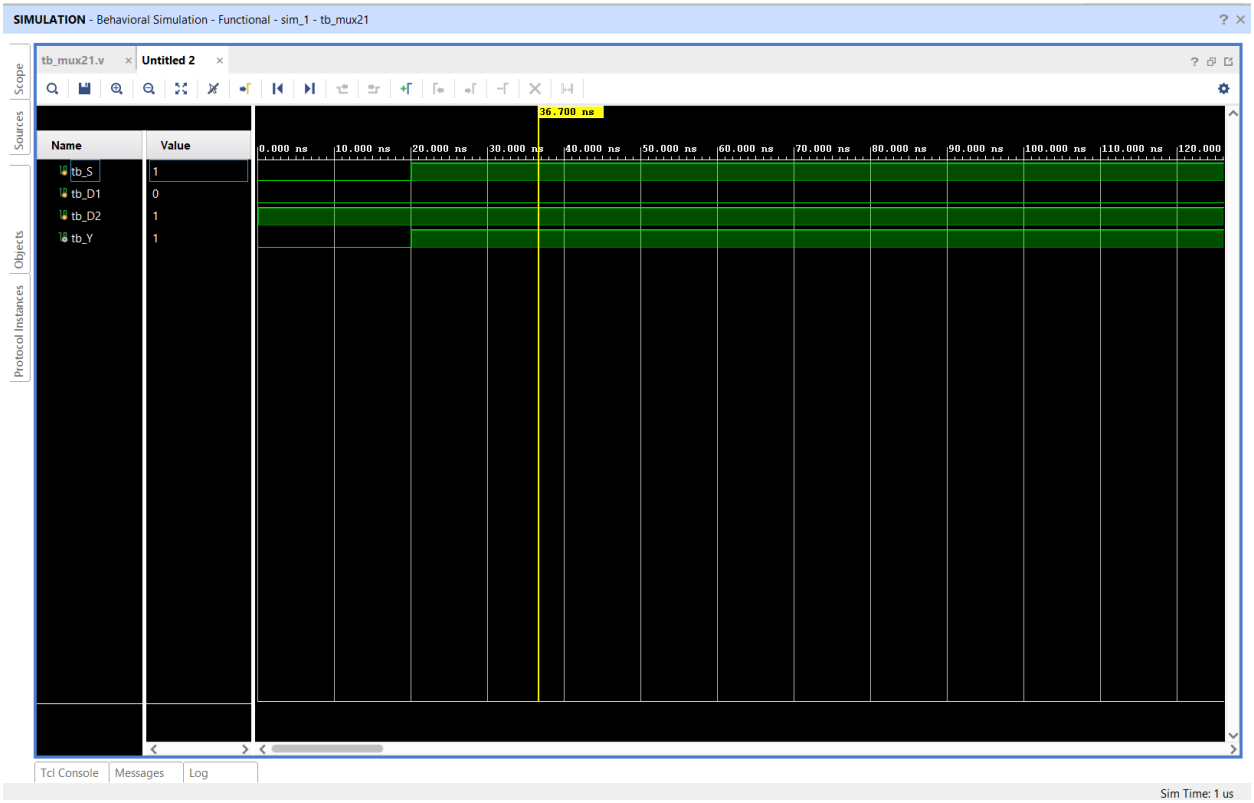
2.Procedure:

The truth table for the 2:1 Multiplexer is shown below, and is used to implement this project: (X=Can take any value 0 or 1, the output is independent of this value)

INPUT S	D1	D2	OUTPUT Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

3.Simulation Results:



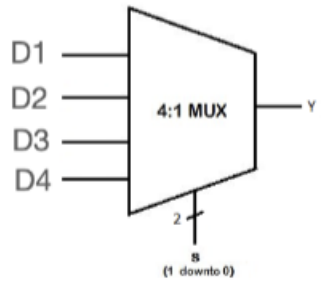


The outputs were as expected, with a single set of values for D1, and D2 were chosen, and the Y value was changed based on the S value input.

Multiplexer 4:1

1.Objective:

Create a 1 Bit , 4:1 Multiplexer, that can act as a switch between four possible 1 Bit inputs, based on a 2 bit input (different from a 1 bit input from the 2:1 multiplexer), following the Block diagram as shown below:



2.Procedure:

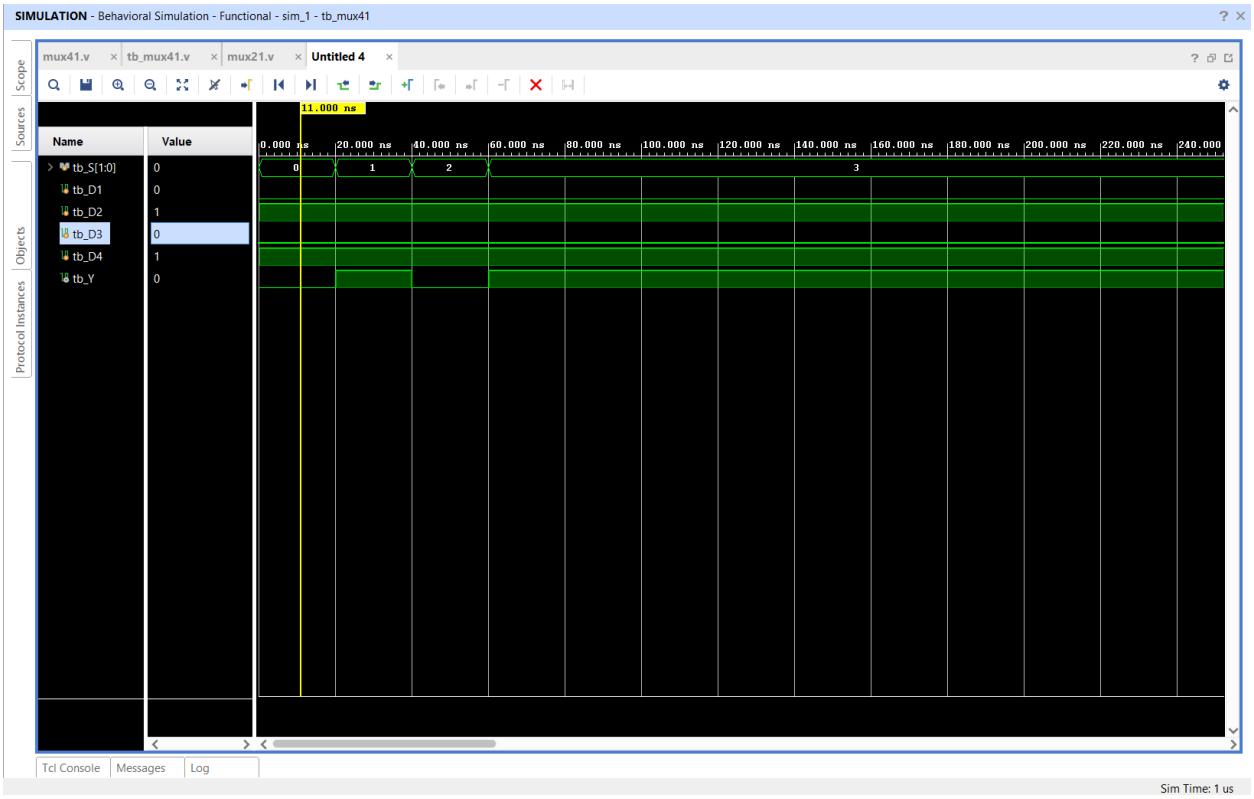
The truth table for the 2:1 Multiplexer is shown below, and is used to implement this project: (X=Can take any value 0 or 1, the output is independent of this value)

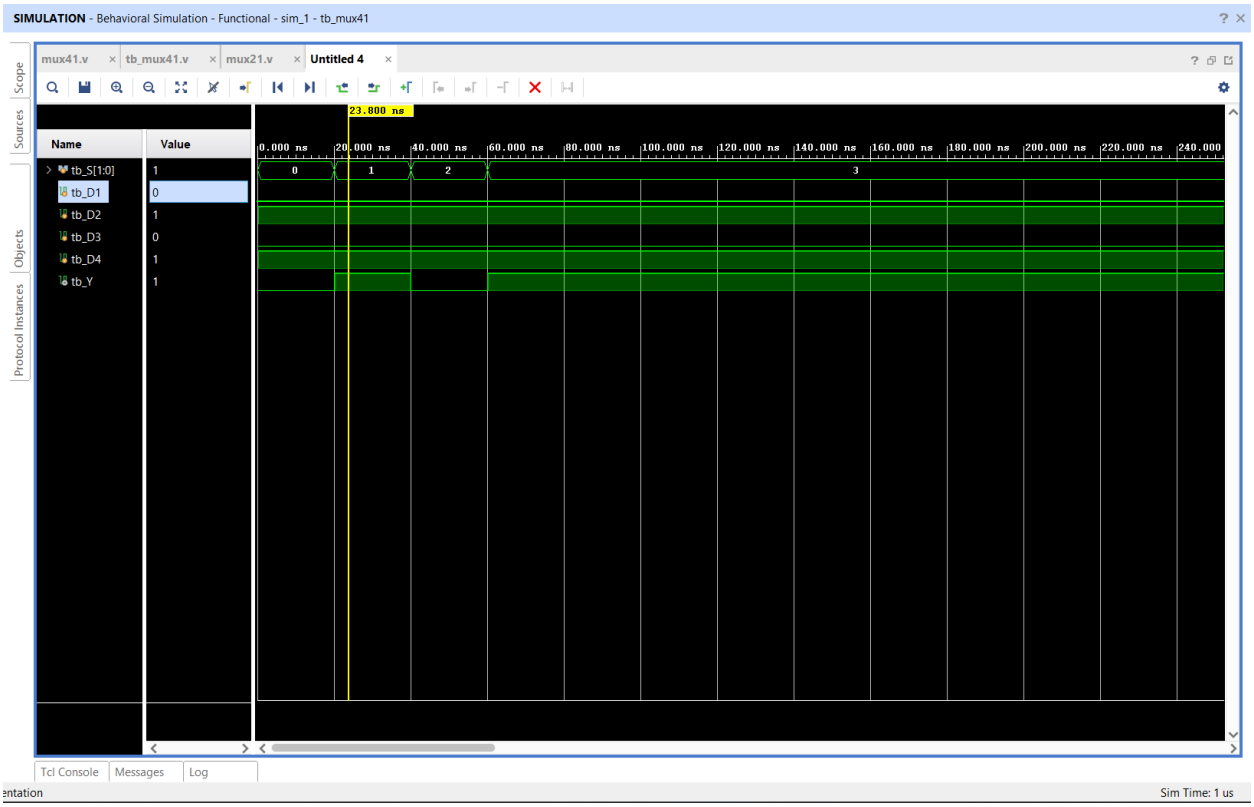
INPUT S[1:0]	D1	D2	D3	D4	OUTPUT Y
00	0	X	X	X	0
00	1	X	X	X	1
01	X	0	X	X	0
01	X	1	X	X	1
10	X	X	0	X	0
10	X	X	1	X	1
11	X	X	X	0	0
11	X	X	X	1	1

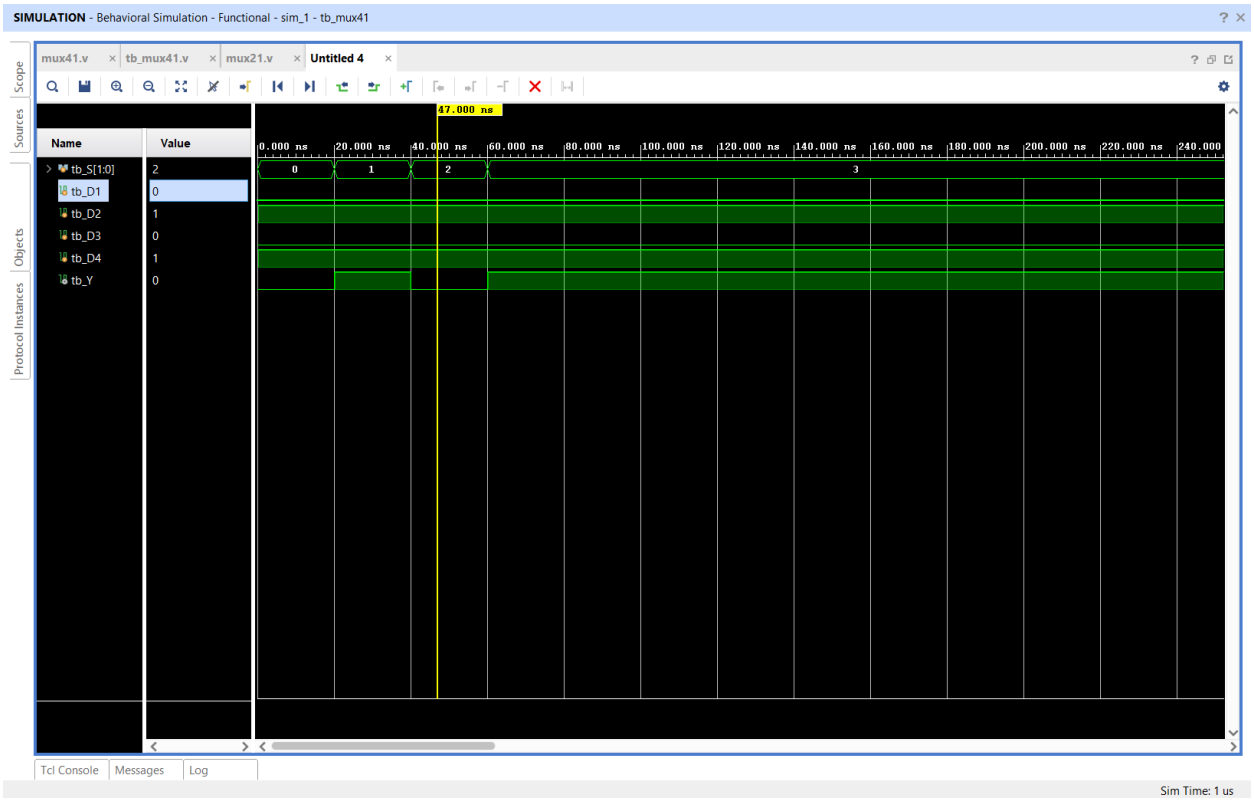
Both the multiplexers were dependent on a conditional operator assigning certain input values to certain output values as seen below:

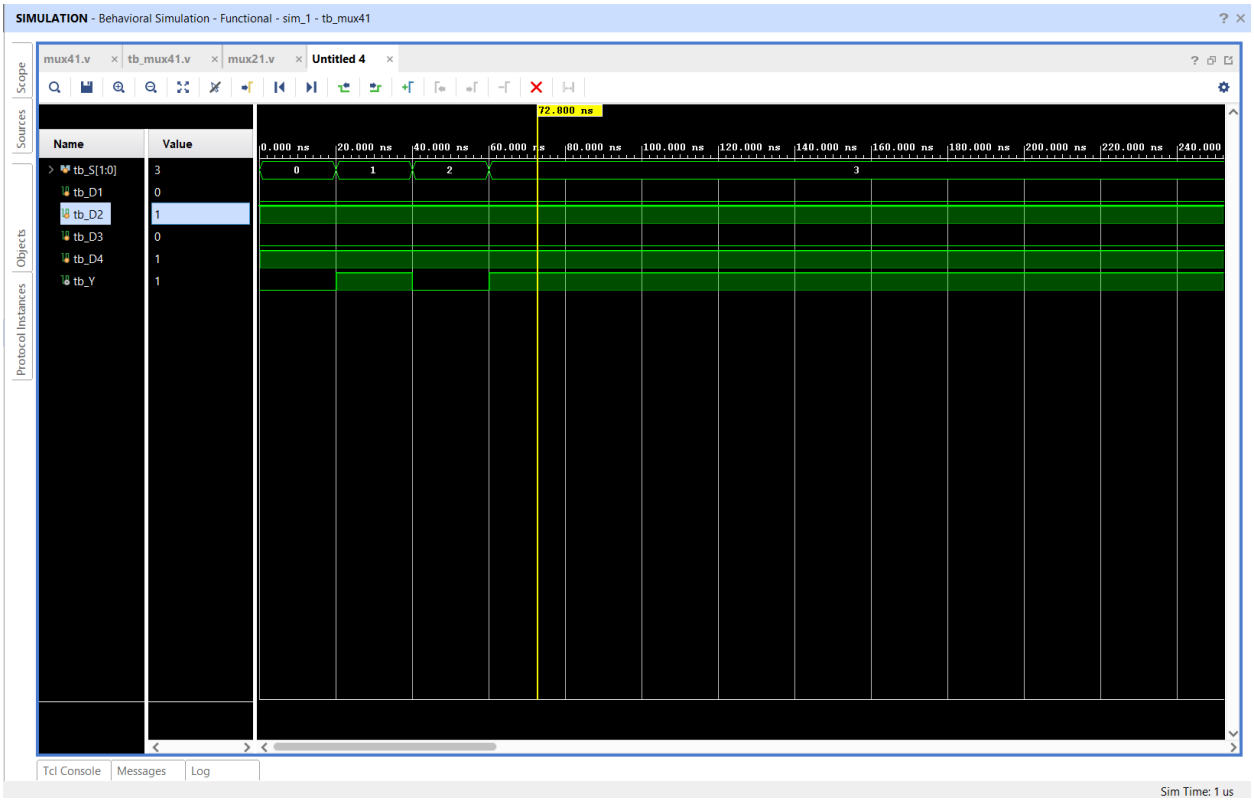
```
Y=(S==2'b00)? D1:
(S==2'b01)? D2:
(S==2'b10)? D3:
D4;
```

3.Simulation Results:




SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_mux41
mux41.v
tb_mux41.v
mux21.v
Untitled 4
23.800 ns
Name
Value
tb_S[1:0]
tb_D1
tb_D2
tb_D3
tb_D4
tb_Y
Tcl Console
Messages
Log
Sim Time: 1 us


SIMULATION - Behavioral Simulation - Functional - sim_1 - tb_mux41
mux41.v
tb_mux41.v
mux21.v
Untitled 4
47.000 ns
Name
Value
tb_S[1:0]
tb_D1
tb_D2
tb_D3
tb_D4
tb_Y
Tcl Console
Messages
Log
Sim Time: 1 us



The outputs were as expected, with a single set of values for D1, and D2 were chosen, and the Y value was changed based on the S value input.